

Lezione 18: Unit Testing con JUnit 5 Jupiter e Assertions

Questo notebook raccoglie tutto il codice della Lezione 18 (`MavenDate`):

- `src/test/java/it/oop/core/TestDate.java` (Test JUnit 5 del costruttore e delle eccezioni)
- `src/test/java/it/oop/core/TestItalianDate.java` (Test JUnit 5 di formattazione con fixture `@BeforeAll`)
- `src/main/java/it/oop/ui/MainStream.java`
- `src/main/java/it/oop/ui/MainDate.java`
- Tutte le classi e eccezioni del progetto `MavenDate`

Struttura dei file della lezione (path dalla cartella radice):

```
Programmazione-II/  
└─ codice-commentato/  
    └─ Lezione18/  
        └─ MavenDate  
            ├── pom.xml  
            └─ src  
                ├── main  
                │   ├── java  
                │   │   └─ it  
                │   │       └─ oop  
                │   │           ├── core  
                │   │           │   ├── AmericanDate.java  
                │   │           │   ├── BirthDay.java  
                │   │           │   ├── Date.java  
                │   │           │   ├── DateInterval.java  
                │   │           │   ├── DatePair.java  
                │   │           │   ├── FormattedDate.java  
                │   │           │   ├── FormattedDateConverter.java  
                │   │           │   ├── ItalianDate.java  
                │   │           │   ├── OrderedPair.java  
                │   │           │   ├── Pair.java  
                │   │           │   ├── Time.java  
                │   │           │   └─ TimeStamp.java  
                │   │           ├── exception  
                │   │           │   ├── IllegalDateException.java  
                │   │           │   └─ OrderdPairException.java  
                │   │           └─ ui  
                │   │               ├── Date.java  
                │   │               ├── MainDate.java  
                │   │               └─ MainStream.java  
                └─ test  
                    ├── java  
                    │   └─ it  
                    │       └─ oop  
                    │           └─ core  
                    │               ├── TestDate.java  
                    │               └─ TestItalianDate.java  
                    └─ resources
```

Argomenti trattati:

- Framework di testing moderno JUnit 5 (Jupiter engine)
- Annotazioni di ciclo di vita del test: `@Test` , `@BeforeAll` , `@BeforeEach`
- Asserzioni di valore: `Assertions.assertEquals(expected, actual)`
- Asserzioni su eccezioni sollevate: `Assertions.assertThrows(ExpectedException.class, executable)`
- Integrazione Maven Surefire Plugin per l'esecuzione automatizzata dei test

1. Classi del Modello e Gestione Eccezioni

```
In [1]: class IllegalDateException extends RuntimeException {
    public IllegalDateException(String message) { super(message); }
}

class Date implements Comparable<Date> {
    protected int day, month, year;
    public Date(int day, int month, int year) {
        this.day = day; this.month = month; this.year = year;
        verify();
    }
    void verify() {
        if (year < 0) throw new IllegalDateException("Illegal date: wrong year");
        if (month < 1 || month > 12) throw new IllegalDateException("Illegal date: wrong month");
        if (day < 1 || day > daysPerMonth(month)) throw new IllegalDateException("Illegal date: wrong
day");
    }
    public int getDay() { return day; }
    public int getMonth() { return month; }
    public int getYear() { return year; }
    public static int daysPerMonth(int month) {
        switch(month) {
            case 4: case 6: case 9: case 11: return 30;
            case 2: return 28;
            default: return 31;
        }
    }
}
@Override public String toString() { return String.format("y%dm%dd%d", year, month, day); }
@Override public int compareTo(Date other) {
    int d = this.year - other.year;
    if (d != 0) return d;
    d = this.month - other.month;
    if (d != 0) return d;
    return this.day - other.day;
}

class ItalianDate extends Date {
    public ItalianDate(int day, int month, int year) { super(day, month, year); }
    public String printFormat() { return "dd/mm/yyyy"; }
    @Override public String toString() { return day + "/" + month + "/" + year; }
}
```

2. Classe di Test `TestDate` ed Esecuzione

```
In [2]: class TestDate {
    private Date date;

    public void testDateConstructor() {
        date = new Date(1, 12, 2025);
        assert date.getDay() == 1 : "Day mismatch";
        assert date.getMonth() == 12 : "Month mismatch";
        assert date.getYear() == 2025 : "Year mismatch";
        System.out.println("testDateConstructor: PASSED"); // testDateConstructor: PASSED
    }

    public void testDateConstructorException() {
        boolean exceptionThrown = false;
        try {
            date = new Date(1, 12, -2);
        } catch (IllegalDateException ide) {
            exceptionThrown = true;
        }
        assert exceptionThrown : "Expected IllegalDateException not thrown";
        System.out.println("testDateConstructorException: PASSED"); // testDateConstructorException:
PASSED
    }

    public static void main(String[] args) {
        TestDate testDate = new TestDate();
        testDate.testDateConstructor();
        testDate.testDateConstructorException();
    }
}
```

```
}  
TestDate.main(null);
```

testDateConstructor: PASSED

testDateConstructorException: PASSED

3. Classe di Test `TestItalianDate` ed Esecuzione

In [3]:

```
class TestItalianDate {  
    private static ItalianDate date;  
  
    public static void setup() {  
        date = new ItalianDate(1, 1, 1970);  
    }  
  
    public void printFormatTest() {  
        setup();  
        assert "dd/mm/yyyy".equals(date.printFormat()) : "Format mismatch";  
        System.out.println("printFormatTest: PASSED"); // printFormatTest: PASSED  
    }  
  
    public static void main(String[] args) {  
        TestItalianDate testIt = new TestItalianDate();  
        testIt.printFormatTest();  
    }  
}  
TestItalianDate.main(null);
```

printFormatTest: PASSED